Developer Guide

Everything a new developer needs to run PhysioGuidance locally, understand the repo layout, write migrations, run tests, and ship code to production from clone to first deploy.

PREPARED FOR

PhysioGuidance Engineering Team

DATE

2026-08-17

DEPLOYED COMMIT

`d396b040`

PHYSIOGUIDANCE

Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

Table of contents

1. Prerequisites
2. Clone and first-time setup
3. Repo layout
4. Running services locally
5. Environment configuration for local
6. Database and migrations (Drizzle)
7. Testing
8. Code style and linting
9. Branching and PR workflow
10. Debugging and logs
11. Shipping to production
12. First-run flow
13. Component status matrix

---

1. Prerequisites

- macOS or Linux
- Node.js 20 (Strapi engines pin `>=20.0.0 <=24.x.x`; nvm recommended)
- Docker Desktop (runs Postgres, Redis, Mailpit, and optional pgAdmin locally)
- Git
- `gcloud` CLI (for deploys and secret access)
- GNU Make (installed by default on macOS and most Linux distros; drives the local `make` targets)
- IAM access on the GCP project: at minimum `roles/run.developer`, `roles/artifactregistry.writer`, `roles/iam.serviceAccountUser`

---

2. Clone and first-time setup

```
git clone https://github.com/physioguidance1-arch/physioguidance.git
cd physioguidance

# Authenticate to GCP for secret access and container pushes
gcloud auth login
gcloud auth application-default login
gcloud auth configure-docker me-central2-docker.pkg.dev --quiet
gcloud config set project nelc-elearning-platform-479319

# Install dependencies per workspace
cd backends && npm install && cd ..
cd frontend && npm install && cd ..
cd strapi   && npm install && cd ..
```

Each workspace has its own `package-lock.json` and installs independently. There is no root-level workspace manifest.

3. Repo layout

```
physioguidance/
├── Makefile                    top-level task runner (run "make help")
├── docker-compose.yml          single compose file for the full local stack
├── README.md
│
├── backends/                   NestJS API + async worker
│   ├── src/
│   │   ├── modules/            45 feature modules (auth, courses, payments, etc.)
│   │   ├── database/
│   │   │   ├── schema/         82 Drizzle ORM schema files
│   │   │   ├── migrations/     122 SQL migrations + meta/_journal.json
│   │   │   ├── seeds/          seed entrypoint + KB seed
│   │   │   ├── seed-roles.ts
│   │   │   └── seed-scfhs-numbers.ts
│   │   ├── common/             shared middleware, guards, validators, crypto
│   │   ├── config/             per-module config loaders
│   │   ├── main.ts             HTTP server entrypoint (Cloud Run API)
│   │   └── main-job.ts         Cloud Run Jobs entrypoint (single-shot workers)
│   ├── scripts/                one-off maintenance and smoke scripts
│   ├── drizzle.config.ts
│   ├── Dockerfile.dev
│   └── package.json
│
├── frontend/                   Next.js 16 (App Router with [locale])
│   ├── src/
│   │   ├── app/[locale]/       pages and layouts (RTL for /ar)
│   │   ├── components/         feature components grouped by dashboard
│   │   ├── lib/                API client and utilities
│   │   └── contexts/           auth, theme, feature flags
│   ├── messages/               en.json + ar.json translations
│   ├── scripts/                check-i18n.js and other build-time helpers
│   ├── public/                 static assets, logos, icons
│   ├── eslint.config.mjs       ESLint 9 flat config
│   ├── next.config.ts
│   └── package.json
│
├── strapi/                     Strapi 5 CMS (blog + marketing content)
│   ├── config/                 Strapi config (DB, plugins, admin)
│   ├── src/                    content-type definitions
│   ├── database/               SQLite dev DB (Postgres in prod)
│   └── package.json
│
├── infra/                      deployment scripts and configuration
│   ├── backend/deploy.sh
│   ├── frontend/deploy.sh
│   ├── worker/deploy.sh
│   ├── strapi/deploy.sh
│   ├── malware-scanner/        includes main.js + deploy.sh
│   ├── monitoring/             Cloud Monitoring dashboard configs
│   ├── scripts/
│   │   ├── migrate-production.sh
```

```
│   │   ├── setup-gcp.sh
│   │   ├── setup-database.sh
│   │   ├── setup-storage.sh
│   │   ├── manage-secrets.sh
│   │   └── ...
│   └── config.sh                shared project, region, and registry vars
│
├── .github/workflows/           4 path-scoped deploy workflows
├── docs/                        internal engineering notes
└── PhysioGuidance-Handover/     client handover bundle (PDFs + CSVs)
```

---

4. Running services locally

The full local stack (frontend, backend, Postgres, Redis, Mailpit) is defined in the root `docker-compose.yml` and driven through the top-level `Makefile`. Run `make help` at any time to see every target.

4.1 Everything at once

```
make dev              # start frontend + backend + database + redis + mailpit
make dev-logs         # follow logs from all services
make dev-down         # stop everything
make dev-restart      # restart everything
```

Once `make dev` reports ready:

- Frontend: `http://localhost:3000`
- Backend API: `http://localhost:8080`
- Postgres: `localhost:5433` (mapped from the container's `5432`)
- Redis: `localhost:6379`
- Mailpit web UI: `http://localhost:8025` (SMTP on `1025`)

4.2 Individual services

```
make dev-frontend     # only the Next.js container
make dev-backend      # backend + database + redis (via depends_on)
make dev-db           # only Postgres
make dev-db-tools     # Postgres + pgAdmin on http://localhost:5050
```

Strapi runs on the host (not in the shared compose file). Start it when you need blog work:

```
cd strapi
npm run develop       # Strapi admin on http://localhost:1337/admin
```

4.3 Local-only behaviors

- `STORAGE_DRIVER=local` in the backend writes uploads to the container volume `backend_uploads` mounted at `/app/uploads`, served through `StorageController` at `GET /api/storage/files/:bucket/*`.
- `StorageService.getScanStatus()` returns `clean` when running against the local storage driver (see `malware-unscanned-file-gate.md` §2). The malware scanner Cloud Run service runs against real GCS buckets only.
- Mailpit captures every outbound email; open `http://localhost:8025` to inspect them.
- Payment gateways read `MOYASAR_MODE=test`, Tamara sandbox URL, and Tabby sandbox keys when `NODE_ENV !== 'production'`.
- Zoom S2S OAuth uses the development app credentials from `docker-compose.yml` (separate from the production app).

---

5. Environment configuration for local

Copy the tracked examples into the ignored working files:

```
cd backends
cp .env.example .env.local
# fill in local Postgres, Redis, and dev secret placeholders

cd ../strapi
cp .env.example .env
# fill APP_KEYS, JWT_SECRET, and encryption keys

cd ../frontend
touch .env.local
# populate NEXT_PUBLIC_API_URL, Strapi URL/token, Sentry DSN, etc.
```

The backend compose service loads `backends/.env.local` via `env_file`, so it must exist before `make dev-backend` or `make dev` starts the backend container. Frontend env vars are read directly from `frontend/.env.local` by Next.js.

Keep `.env`, `.env.local`, and any variant untracked. `.gitignore` covers them by default; verify any new file you add stays out of `git status`.

The full production env-var reference lives in environment-configuration-CONFIDENTIAL.md.

---

6. Database and migrations (Drizzle)

- ORM: Drizzle (`backends/src/database/schema/*.ts` , TypeScript-first schema definition, 82 schema files).
- Migration tool: `drizzle-kit` . `backends/drizzle.config.ts` sets `dialect: 'postgresql'` , points at `src/database/schema/index.ts` , and writes migrations under `src/database/migrations` .
- Migration journal: `backends/src/database/migrations/meta/_journal.json` . Drizzle skips any migration not registered in the journal, so it must always be committed alongside the SQL file (122 entries and counting).

6.1 Available commands

Backend `package.json` :

Script	Command	Purpose
`db:generate`	`drizzle-kit generate`	Diff schema, write a numbered SQL migration, update `_journal.json`
`db:migrate`	`drizzle-kit migrate`	Apply pending migrations tracked in `__drizzle_migrations`
`db:push`	`drizzle-kit push`	Sync schema directly (dev only, no migration file)
`db:studio`	`drizzle-kit studio`	Open Drizzle Studio at `https://local.drizzle.studio`
`db:seed`	`ts-node src/database/seeds/index.ts`	Populate demo data
`db:seed:roles`	`ts-node src/database/seed-roles.ts`	Insert canonical role rows
`db:seed:scfhs`	`ts-node src/database/seed-scfhs-numbers.ts`	Load SCFHS professional identifiers
`db:seed:kb`	`ts-node src/database/seeds/seed-kb.ts`	Seed the knowledge base
`db:reset`	`db:push && db:seed:roles`	Wipe schema and reseed roles

Convenience wrappers in the Makefile (`make db-generate` , `make db-migrate` , `make db-push` , `make db-studio` , `make db-seed` , `make db-reset`) shell into the running backend container.

6.2 Add a migration

```
cd backends

# 1. Edit the relevant schema file under src/database/schema/
#    Add a column, table, index, whatever.

# 2. Generate the migration
npm run db:generate
# Creates NNNN_name.sql under src/database/migrations/ AND updates meta/_journal.json

# 3. Verify the journal entry landed
git diff src/database/migrations/meta/_journal.json

# 4. Apply locally
npm run db:migrate

# 5. Commit the .sql file, the snapshot under meta/, and _journal.json together
```

6.3 Apply migrations to production

Always via `infra/scripts/migrate-production.sh` (or `make migrate-prod`). The script:

1. Loads `infra/config.sh` for project, region, and Cloud SQL instance name.
2. Fetches the `DATABASE_URL` secret from Secret Manager.
3. Starts `cloud-sql-proxy` on `localhost:5433` against the private-IP Cloud SQL instance, rewriting the URL host so Drizzle connects through the proxy.
4. Runs `npm run db:migrate` from the `backends/` workspace so `__drizzle_migrations` stays authoritative.
5. Tears the proxy down when done.

```
./infra/scripts/migrate-production.sh
# or
make migrate-prod
```

Migrations always flow through this path so `__drizzle_migrations` remains the single source of truth for what has been applied.

6.4 Schema highlights

- 82+ Drizzle schema files organised by domain.
- Foreign keys enforced (see the `pdpl-inactive-user-purger` job that walks 68 FK cascades on user hard-delete).
- Timestamps: every table has `created_at`; most have `updated_at`.
- Soft delete: `users.deleted_at` (drives the PDPL 30-day grace window).
- Encrypted columns: `refund_iban`, `refund_national_id`; see security-architecture.md §10.

7. Testing

7.1 Backend

- Framework: Jest, configured inline in `backends/package.json` (`rootDir: src` , pattern `*.spec.ts` , transformer `ts-jest` , `testEnvironment: node`).
- Existing suites: 12 `*.spec.ts` files covering crypto, distributed lock, session reminders, task naming, SLA monitor, points cron, scheduled maintenance, and the payment gateways (Tamara, Tabby, mock, invoice, Tamara retry).
- Commands:

```
cd backends
npm test                # jest
npm run test:watch      # jest --watch
npm run test:cov        # jest --coverage
```

7.2 Frontend

- Framework: Jest 30 with `jest-environment-jsdom` , `@testing-library/react` , and `@testing-library/jest-dom` on the dev-dependency list.
- Commands:

```
cd frontend
npm test                # jest
npm run test:coverage   # jest --coverage
npx tsc --noEmit        # TypeScript type check
```

7.3 i18n guard

```
cd frontend
npm run check:i18n
```

`frontend/scripts/check-i18n.js` walks every `t("key.path")` call in `src/` , resolves the namespace from `useTranslations("namespace")` , and confirms the key exists in both `messages/en.json` and `messages/ar.json` . It also reports keys present in one locale file but not the other. The `build` script runs it first, so a Next.js production build fails fast when a translation is missing.

8. Code style and linting

8.1 Backend

```
cd backends
npm run lint     # eslint "{src,apps,libs,test}/**/*.ts" --fix
npm run format   # prettier --write "src/**/*.ts"
```

Configuration: `@typescript-eslint/eslint-plugin`, `eslint-plugin-prettier`, and `eslint-config-prettier` (see `backends/package.json` devDependencies). TypeScript runs in strict mode.

8.2 Frontend

```
cd frontend
npm run lint     # eslint (flat config in eslint.config.mjs, extends next core web vitals)
```

8.3 Naming conventions

- PascalCase for classes, types, React components
- camelCase for functions and variables
- kebab-case for file names (React component files stay PascalCase)
- SCREAMING_SNAKE for env vars and top-level constants

---

9. Branching and PR workflow

- Default production branch: `main`. Path-scoped GitHub Actions workflows deploy automatically on merge (see cicd-deployment.md §3).
- Feature branches fork from `main`; PRs target `main`.
- Trunk-based: there is no long-lived `develop` branch.
- Docs-only edits under `PhysioGuidance-Handover/**` do not touch the paths that trigger the deploy workflows (`backends/**`, `frontend/**`, `strapi/**`, `infra/**`).

---

10. Debugging and logs

- Local dev: services log to stdout via Nest's built-in `Logger` (colorised) and Next.js's dev logger. Mailpit at `http://localhost:8025` captures outbound email.

- Production logs: GCP Console → Cloud Logging → filter by resource `Cloud Run Revision` and the service name.
- Real-time tail:

```
gcloud run services logs tail physioguidance-api --region=me-central2
```

- Errors: land in Sentry (`@sentry/nestjs` on the backend, `@sentry/nextjs` on the frontend). Open the issue detail for full stack, breadcrumbs, and request context.
- Database traces: set `LOG_LEVEL=debug` on the Cloud Run service to surface slow-query traces; revert after debugging.

---

11. Shipping to production

Full flow lives in cicd-deployment.md. Short version:

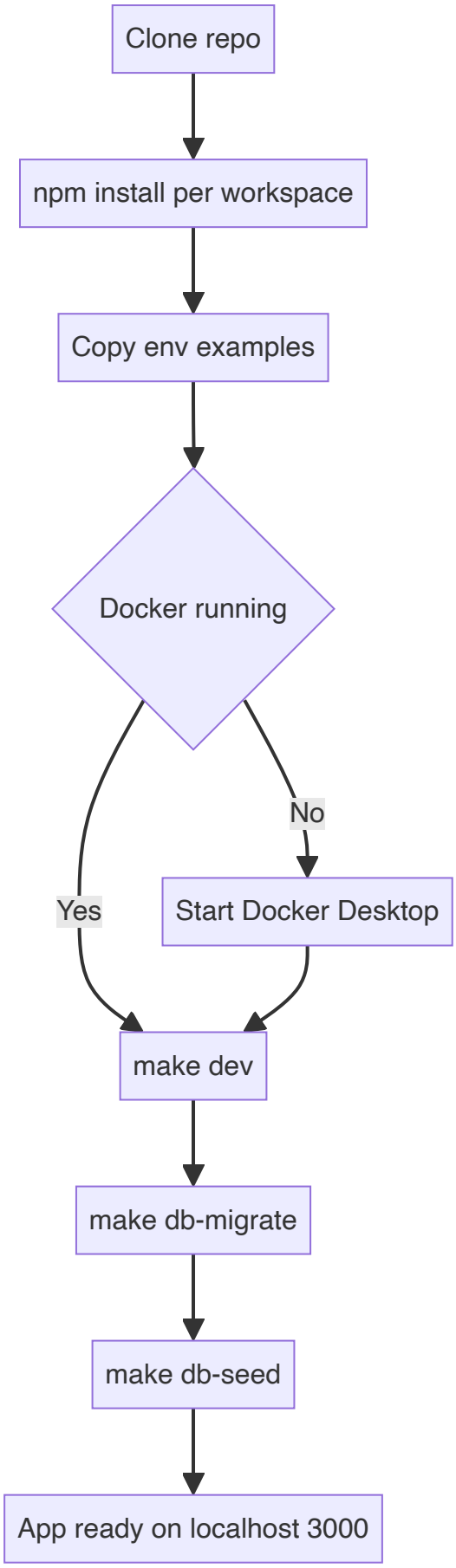
1. Merge to `main` .
2. Path-scoped GitHub Actions fire: `deploy-backend.yml` , `deploy-frontend.yml` , `deploy-strapi.yml` . `deploy-worker.yml` chains from the successful backend workflow so the worker reuses the freshly built image.
3. Each workflow authenticates via Workload Identity Federation, builds through the matching `infra/*/deploy.sh` script, pushes to Artifact Registry, and deploys to Cloud Run in `me-central2` .
4. Watch the run: `gh run watch` or GCP Console → Cloud Run → Revisions.
5. Verify: `curl https://<service-url>/api/health/ready` .

Rollback: GCP Console → Cloud Run → service → Revisions → route 100% of traffic to the prior revision.

Manual deploy targets are also available: `make deploy-backend` , `make deploy-frontend` , `make deploy-strapi` , `make deploy-all` .

---

12. First-run flow


Clone repo
npm install per workspace
Copy env examples
Docker running
No
Yes
Start Docker Desktop
make dev
make db-migrate
make db-seed
App ready on localhost 3000

13. Component status matrix

Item	Status	Notes
Repo cloneable with `npm install` per workspace	Implemented	3 workspaces: `backends`, `frontend`, `strapi`
Local dev via Docker Compose	Implemented	Root `docker-compose.yml` driven by `make dev`
Makefile task runner	Implemented	`make help` lists every dev, DB, and deploy target
`.env.example` templates	Implemented	`backends/.env.example` and `strapi/.env.example`; frontend uses `.env.local`
Drizzle migration workflow	Implemented	`db:generate`, `_journal.json`, and `migrate-production.sh` behind cloud-sql-proxy
CI-enforced i18n check	Implemented	`check:i18n` runs pre-build; missing EN or AR keys fail the frontend build
Backend unit tests + coverage	Implemented	Jest with `test`, `test:watch`, `test:cov` (12 spec files)
Frontend unit test harness	Implemented	Jest 30, jsdom, testing-library, `test` and `test:coverage` scripts
Backend lint + format	Implemented	`npm run lint` (ESLint --fix) and `npm run format` (Prettier)
Path-scoped GitHub Actions deploys	Implemented	4 workflows targeting Cloud Run in `me-central2`

14. Where to look next

- How CI deploys: cicd-deployment.md
- Every env var and secret: environment-configuration-CONFIDENTIAL.md
- Every API endpoint: api-reference.md
- Ops runbook for on-call: ops-runbook.md